



ENSIM
École d'ingénieurs
Le Mans Université



Compte Rendu de TP Instrumentation Virtuelle : Acquisition automatisée de réponses en fréquence

ESSOMBA JABE MICHAEL MARIE, ACHIBAT ISMAIL

7 janvier 2026

Tuteur(s) université :
MALANDIN EMMANUEL



**Le Mans
Université**

9 janvier 2026

Table des matières

Introduction générale	3
I Instrumentation directe via VISA/GPIB	5
1 Objectifs et principe de mesure	5
1.1 Objectifs pédagogiques et techniques	5
1.2 Montage expérimental et instruments utilisés	5
1.3 Principe physique de la mesure	7
1.4 Montage expérimental et chaîne de mesure	7
1.5 Balayage fréquentiel	8
1.6 Conversion amplitude crête-à-crête vers valeur RMS	8
1.7 Notion de fréquence de coupure	8
1.8 Description de l'interface utilisateur	9
2 Implémentation logicielle – Instrumentation directe	10
2.1 Initialisation et configuration des instruments	10
2.2 Balayage fréquentiel et acquisition des mesures	11
2.3 Détermination automatique de la fréquence de coupure	11
3 Résultats et analyse – Instrumentation directe	12
3.1 Discussion des résultats	13
II Architecture Client–Serveur TCP/IP	14
4 Objectifs et architecture générale	14
4.1 Objectifs de la mise en architecture distribuée	14
4.2 Principe de l'architecture retenue	14
5 Interfaces Client et Serveur	15
5.1 Interface Serveur : rôle et choix des composants	15
5.2 Interface Client : rôle et choix des composants	15
5.3 Conception et rôle des interfaces Client et Serveur	16
5.4 Illustration des interfaces réalisées	17
6 Principe général des échanges	19
7 Requête envoyée par le client	19



8 Réponse envoyée par le serveur	19
9 Cycle de fonctionnement du serveur	20
10 Acquisition et envoi des données	20
11 Journalisation et supervision côté serveur	20
12 Implémentation côté client	21
12.1 Gestion de la connexion TCP	21
12.2 Échanges réseau et journalisation côté client	21
12.3 Réception des données et traitement	22
12.4 Calcul de la fréquence de coupure côté client	22
12.5 Affichage des résultats	23
13 Bilan et améliorations	23
13.1 Bilan	23
13.2 Difficultés rencontrées	23
13.3 Améliorations possibles	24
Conclusion générale	24
A Annexe A – Code source (Partie 1)	24
A.1 Fichier TP1C.txt (complet)	24
B Annexe B – Code source (Serveur TCP/VISA)	33
B.1 Fichier ServeurC.txt (complet)	33
C Annexe C – Code source (Client TCP/UI)	37
C.1 Fichier clientC.txt (complet)	37

Introduction générale

Contexte et objectifs du TP

Ce travail pratique s'inscrit dans le cadre de l'**instrumentation virtuelle**, dont le principe est de piloter des instruments de mesure réels à l'aide d'un logiciel, afin d'automatiser les acquisitions, de traiter les données et de visualiser les résultats.

Le banc expérimental utilisé est composé principalement de :

- un **GBF** (générateur basse fréquence) qui fournit une sinusoïde d'entrée $V_{in}(t)$,
- un **multimètre numérique** qui mesure la tension efficace en sortie $V_{out,rms}$,
- un **système à caractériser** (filtre analogique, typiquement de type RC).

L'objectif technique est de mesurer automatiquement la **réponse en fréquence** du filtre, c'est-à-dire l'évolution du gain en fonction de la fréquence. On obtient ainsi un **diagramme de Bode (module)** représentant :

$$G_{dB}(f) = 20 \log_{10} \left(\frac{V_{out,rms}(f)}{V_{in,rms}(f)} \right)$$

Un point clé de l'analyse consiste à estimer la **fréquence de coupure** f_c , définie comme la première fréquence pour laquelle le gain descend sous le seuil :

$$G(f_c) = G_{\max} - 3 \text{ dB}$$

Cette fréquence permet de valider qualitativement et quantitativement le comportement attendu du filtre (bande passante, atténuation, pente au-delà de f_c , etc.).

Démarche générale

Le TP est structuré en deux étapes complémentaires :

— Partie 1 : Instrumentation directe

Une application unique (sous LabWindows/CVI) regroupe l'interface utilisateur, la communication VISA/GPIB et le traitement des mesures. L'utilisateur saisit les paramètres (A , $f_{\min}$, $f_{\max}$, N), l'application pilote le GBF, interroge le multimètre, puis trace le diagramme de Bode et calcule f_c .

— Partie 2 : Architecture Client–Serveur

Le contrôle des instruments est déporté dans une application **Serveur** (accès VISA), tandis que l'interface et l'affichage sont assurés par une application **Client**. Les deux programmes communiquent via **TCP/IP**. Le client envoie une requête de mesure, le serveur réalise l'acquisition et renvoie les couples (f, G_{dB}) pour affichage et calcul des résultats.

Cette séparation reproduit un contexte réaliste : un poste « serveur » centralise les instruments, tandis que plusieurs postes clients peuvent potentiellement accéder aux mesures à distance.

Technologies et outils utilisés

- **Environnement de développement** : LabWindows/CVI
- **Langage** : C (ANSI C)
- **Pilotage des instruments** : VISA (commandes SCPI via GPIB)
- **Communication réseau** : TCP/IP (architecture Client–Serveur)



- **Interface graphique (IHM)** : contrôles CVI (Numeric, Knob/Dial, Graph, Ring-Meter, TextBox de logs)

Organisation du document

Le rapport est organisé afin de suivre la progression du projet :

- **Partie 1** : mise en place de l'acquisition automatique en instrumentation directe (VISA/GPIB), traitement et validation des résultats ($G(f)$, f_c).
- **Partie 2** : extension en Client-Serveur TCP/IP : protocole d'échange, implémentation serveur (acquisition VISA), implémentation client (requête, réception, tracé, calculs).
- **Annexes** : codes sources complets et captures d'écran permettant de relier l'analyse aux implémentations réalisées.

Première partie

Instrumentation directe via VISA/GPIB

1 Objectifs et principe de mesure

1.1 Objectifs pédagogiques et techniques

Cette première partie du TP vise à mettre en œuvre une application complète d'instrumentation virtuelle, dans laquelle le logiciel joue un rôle central : il pilote les instruments de mesure, réalise l'acquisition des données et en assure le traitement automatique.

Les objectifs poursuivis sont à la fois pédagogiques et techniques :

- Mettre en œuvre la communication instrumentale via l'interface **VISA/GPIB**.
- Piloter un générateur basse fréquence (GBF) pour produire un signal sinusoïdal d'amplitude contrôlée.
- Effectuer un **balayage fréquentiel automatique** entre une fréquence minimale $f_{\min}$ et une fréquence maximale $f_{\max}$.
- Mesurer, pour chaque fréquence, la tension efficace de sortie $V_{out,rms}$ à l'aide d'un multimètre numérique.
- Calculer le **gain fréquentiel** du système étudié.
- Tracer automatiquement le **diagramme de Bode (module)**.
- Déterminer de manière algorithmique la **fréquence de coupure** f_c correspondant à une chute de -3 dB.

Cette première partie constitue la base fonctionnelle du projet. Les principes développés ici (sweep, calcul du gain, détection de f_c) seront ensuite réutilisés et étendus dans la seconde partie consacrée à l'architecture Client-Serveur.

1.2 Montage expérimental et instruments utilisés

Le montage expérimental repose sur un banc d'instrumentation classique permettant la caractérisation fréquentielle d'un filtre analogique passif. Il associe un générateur de signaux, un instrument de mesure et un montage sur plaque d'essai, l'ensemble étant piloté par logiciel.

Instruments de mesure Deux instruments de laboratoire sont utilisés et pilotés à distance via l'interface VISA/GPIB :

- **Générateur Basse Fréquence (GBF)** : utilisé pour générer un signal sinusoïdal d'amplitude et de fréquence contrôlées. Il constitue la source d'excitation du filtre.
- **Multimètre numérique (DMM)** : utilisé pour mesurer la tension efficace (RMS) en sortie du filtre, en régime sinusoïdal établi.

Les instruments sont reliés au PC par un bus GPIB et identifiés par leurs adresses VISA, configurables depuis l'interface utilisateur :

- **GBF** : GPIB3::10::INSTR
- **Multimètre** : GPIB3::23::INSTR

Ces adresses sont affichées et modifiables dans l'interface afin d'assurer la cohérence entre la configuration matérielle réelle et le pilotage logiciel.

Montage électronique Le circuit étudié est un filtre RC passif réalisé sur une plaque d'essai (breadboard). Il est composé des éléments suivants :

- une résistance de valeur $R = 560 \, \Omega$,
- un condensateur de valeur $C = 470 \, \text{nF}$.

Le signal sinusoïdal fourni par le GBF est appliqué à l'entrée du filtre. La tension de sortie est mesurée aux bornes du condensateur à l'aide du multimètre, conformément à la configuration d'un filtre passe-bas du premier ordre.

Principe de connexion

- La sortie du GBF est reliée à l'entrée du filtre RC.
- Le multimètre est connecté en parallèle sur la sortie du filtre pour mesurer $V_{out,rms}$.
- Les masses des instruments sont communes afin d'assurer une référence électrique cohérente.

Ce montage permet d'observer l'atténuation progressive du signal lorsque la fréquence augmente, et donc de déterminer expérimentalement la fréquence de coupure du filtre.

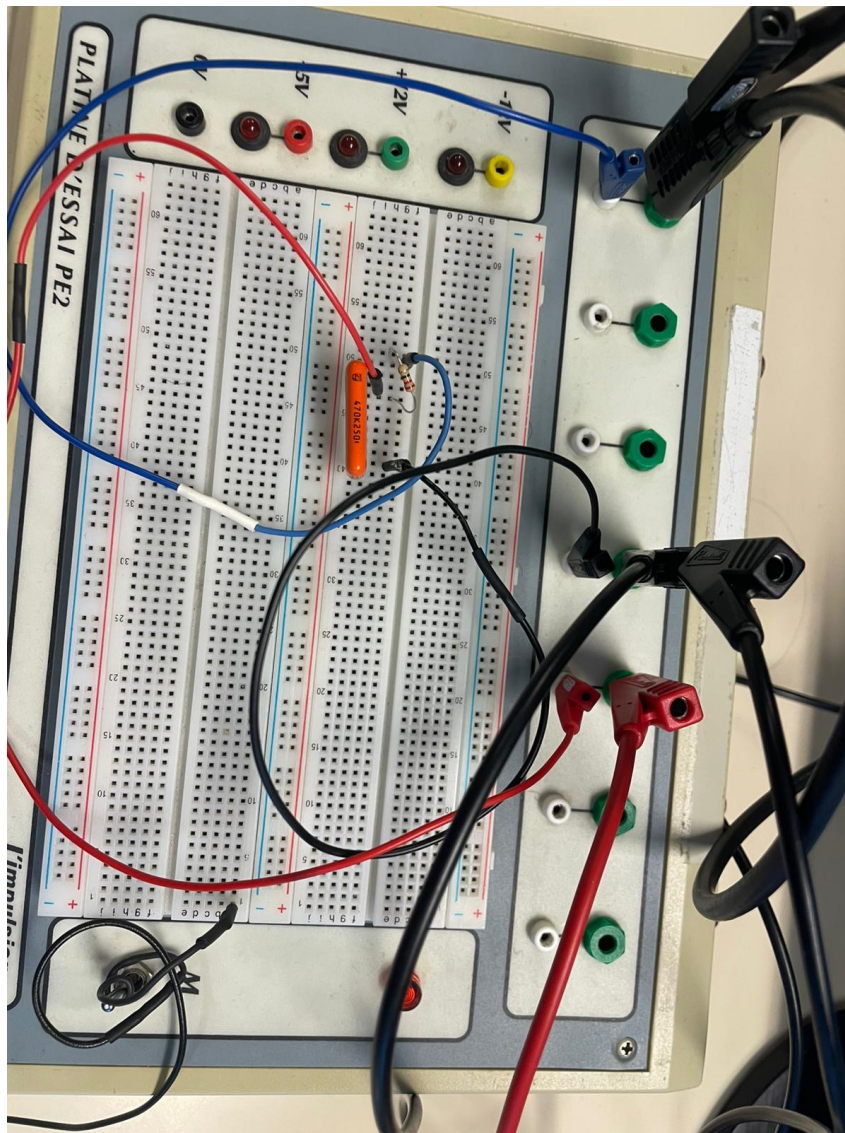


FIGURE 2 – Montage expérimental : filtre RC réalisé sur plaque d'essai, connecté au GBF (entrée) et au multimètre (mesure de la tension de sortie).

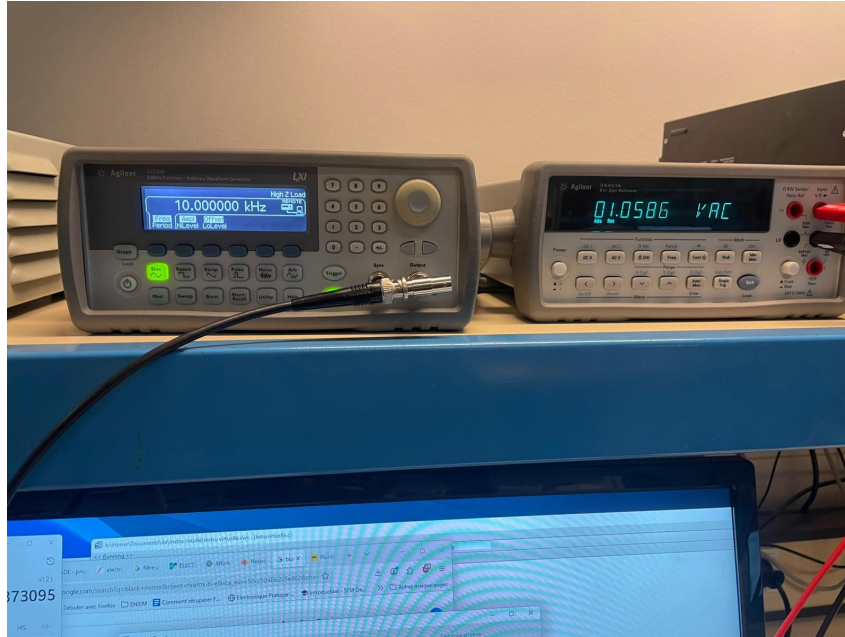


FIGURE 3 – Instruments de mesure utilisés : générateur basse fréquence (à gauche) et multimètre numérique (à droite).

1.3 Principe physique de la mesure

Le système étudié est un filtre analogique (généralement de type RC). Le principe de mesure repose sur l'analyse de la réponse du filtre à un signal sinusoïdal dont la fréquence est progressivement modifiée.

Pour une fréquence donnée f :

- le GBF fournit une tension sinusoïdale $V_{in}(t)$,
- le filtre modifie l'amplitude du signal selon sa fonction de transfert,
- le multimètre mesure la valeur efficace $V_{out,rms}(f)$.

Le gain fréquentiel est alors défini par :

$$G_{dB}(f) = 20 \log_{10} \left(\frac{V_{out,rms}(f)}{V_{in,rms}} \right)$$

Dans l'application développée, cette relation est directement implémentée dans le code, par exemple :

```
1 gainDB = 20.0 * log10(vOut_rms / Vin_rms);
```

L'automatisation de ces opérations permet d'obtenir rapidement une caractérisation complète du filtre sur toute la bande de fréquences étudiée.

1.4 Montage expérimental et chaîne de mesure

Le banc de mesure repose sur trois éléments principaux :

- un **GBF**, piloté par logiciel, qui génère le signal d'entrée,
- le **filtre analogique** à caractériser,
- un **multimètre numérique** chargé de mesurer la tension efficace en sortie.

La chaîne de mesure peut être résumée de la manière suivante :

GBF (signal sinusoïdal) → Filtre analogique → Multimètre (mesure $V_{out,rms}$)
Pilotage et acquisition via VISA/GPIB

FIGURE 4 – Chaîne de mesure utilisée pour l'acquisition de la réponse en fréquence

Le pilotage du GBF (fréquence, amplitude, activation de la sortie) et la lecture des mesures du multimètre sont réalisés à l'aide de commandes SCPI envoyées via l'interface VISA.

1.5 Balayage fréquentiel

Afin d'obtenir un diagramme de Bode exploitable et conforme aux pratiques en électronique analogique, le balayage fréquentiel est effectué en **échelle logarithmique**. Ce choix présente plusieurs avantages :

- meilleure représentation des variations de gain sur plusieurs décades,
- densité de points plus pertinente aux basses fréquences,
- lecture facilitée de la fréquence de coupure.

La fréquence au point i du balayage est calculée par :

$$f_i = f_{\min} \left(\frac{f_{\max}}{f_{\min}} \right)^{\frac{i}{N-1}}$$

Cette relation est directement implémentée dans la boucle de sweep du programme :

```
1 double f = fLow * pow(fHigh / fLow,  
2 (double)i / (nStepsInt - 1));
```

À chaque itération, la fréquence est envoyée au GBF via la commande SCPI FREQ.

1.6 Conversion amplitude crête-à-crête vers valeur RMS

Dans l'interface utilisateur, l'amplitude du signal est définie en tension crête-à-crête (**V_{pp}**). En revanche, le multimètre fournit une mesure en valeur efficace (RMS). Il est donc indispensable de convertir l'amplitude d'entrée afin de garantir un calcul cohérent du gain.

Pour un signal sinusoïdal :

$$V_{in,rms} = \frac{V_{pp}}{2\sqrt{2}}$$

Cette conversion est réalisée une seule fois avant le sweep :

```
1 double Vin_rms = amplitude / (2.0 * sqrt(2.0));
```

Cette valeur est ensuite utilisée pour le calcul du gain à chaque fréquence.

1.7 Notion de fréquence de coupure

La fréquence de coupure f_c est définie comme la fréquence pour laquelle le gain chute de 3 dB par rapport à son maximum :

$$G(f_c) = G_{\max} - 3 \text{ dB}$$

Dans l'application, cette fréquence est déterminée automatiquement selon l'algorithme suivant :

1. recherche du gain maximal $G_{\max}$,
2. calcul du seuil $G_{\max} - 3$,
3. détection du premier point de fréquence pour lequel le gain devient inférieur ou égal à ce seuil.

Cette logique est implémentée directement dans le code :

```
1 double gainMax = gainArray[0];
2 for (int i = 1; i < nValid; ++i)
3     if (gainArray[i] > gainMax) gainMax = gainArray[i];
4
5 double seuil = gainMax - 3.0;
6
7 for (int i = 0; i < nValid; ++i)
8     if (gainArray[i] <= seuil) {
9         fc = freqArray[i];
10        break;
11    }
```

Cette méthode garantit une estimation reproductible de la fréquence de coupure, indépendante de toute observation manuelle du graphe.

1.8 Description de l'interface utilisateur

L'interface graphique développée sous LabWindows/CVI permet à l'utilisateur de configurer entièrement l'acquisition de la réponse en fréquence, de lancer la mesure et de visualiser les résultats de manière claire et synthétique.

Elle est organisée en plusieurs zones fonctionnelles distinctes :

- **Zone de réglage des paramètres du signal** : Cette zone regroupe les contrôles permettant de définir les caractéristiques du signal généré par le GBF :
 - l'amplitude du signal sinusoïdal,
 - le nombre de pas du balayage fréquentiel,
 - la fréquence basse $f_{\min}$,
 - la fréquence haute $f_{\max}$.

Les paramètres sont ajustables à l'aide de boutons rotatifs (knobs) et de champs numériques associés, ce qui permet un réglage précis et intuitif.

- **Zone de configuration des instruments** : Cette partie permet de renseigner les adresses VISA/GPIB du multimètre et du GBF. Elle rend l'application adaptable à différents bancs de mesure sans modification du code.
- **Indicateur de fréquence en cours** : Un indicateur de type jauge (ring meter) affiche la fréquence actuellement appliquée par le GBF durant le balayage. Cet élément permet de suivre visuellement l'avancement de l'acquisition en temps réel.
- **Zone d'affichage graphique** : La partie centrale de l'interface est occupée par un graphe intitulé « *Réponse en fréquence* ». Il affiche le diagramme de Bode du module, représentant le gain en décibels en fonction de la fréquence (échelle logarithmique). Un marqueur est utilisé pour mettre en évidence la fréquence de coupure déterminée automatiquement.

- **Zone d’affichage des résultats numériques** : Des champs dédiés affichent les résultats calculés à l’issue de l’acquisition :
 - la fréquence de coupure f_c ,
 - le gain associé à cette fréquence,
 - la tension efficace de sortie $V_{out,rms}(f_c)$.
- **Commande de lancement** : Un bouton permet de lancer l’acquisition une fois tous les paramètres correctement configurés.

L’ensemble de l’interface a été conçu pour offrir une lecture claire des résultats et un enchaînement logique des opérations, depuis la configuration du signal jusqu’à l’analyse finale de la réponse en fréquence.

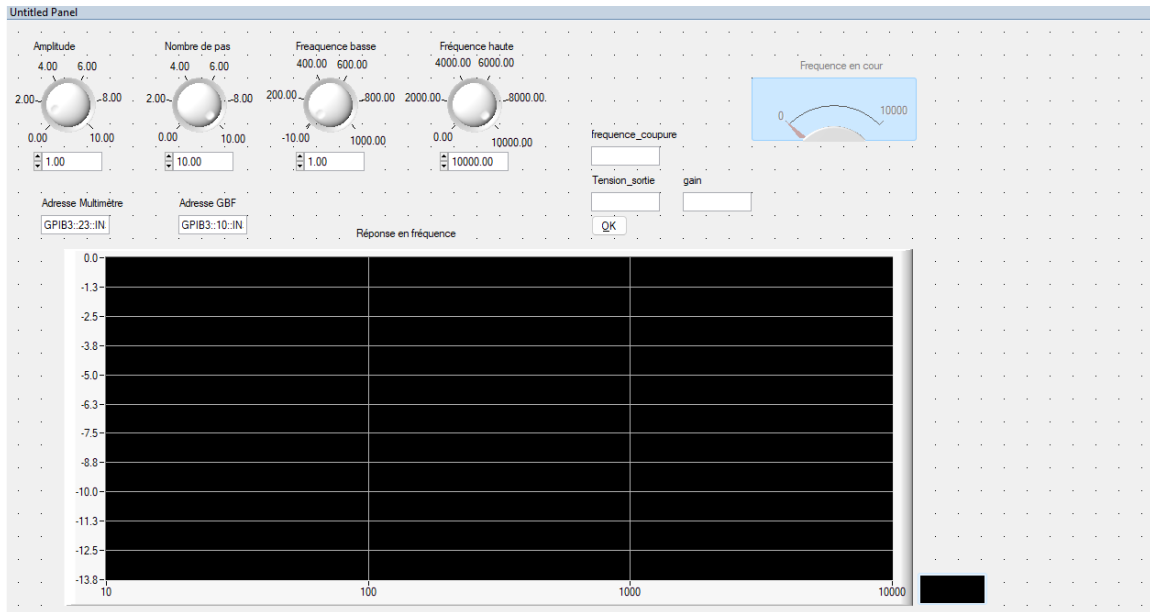


FIGURE 5 – Interface utilisateur réalisée sous LabWindows/CVI pour l’acquisition de la réponse en fréquence

2 Implémentation logicielle – Instrumentation directe

Cette partie décrit l’implémentation logicielle de l’application d’instrumentation directe. Le programme assure successivement l’initialisation des instruments, l’acquisition automatique des mesures lors d’un balayage fréquentiel, puis le traitement des données pour en extraire les grandeurs caractéristiques du filtre.

2.1 Initialisation et configuration des instruments

Au démarrage de l’acquisition, l’application initialise le *Resource Manager* VISA, puis ouvre les sessions de communication avec le générateur basse fréquence (GBF) et le multimètre numérique à partir des adresses GPIB fournies dans l’interface.

Une fois les connexions établies, le GBF est configuré via des commandes SCPI afin de :

- sélectionner une forme d’onde sinusoïdale,
- fixer l’amplitude du signal,
- activer la sortie du générateur.

Cette phase garantit que le système de mesure est correctement initialisé avant le lancement du balayage fréquentiel.

2.2 Balayage fréquentiel et acquisition des mesures

L'acquisition repose sur une boucle de balayage fréquentiel en échelle logarithmique. À chaque itération, les opérations suivantes sont effectuées :

1. calcul de la fréquence courante f_i ,
2. envoi de la commande de fréquence au GBF,
3. délai de stabilisation pour atteindre le régime permanent,
4. mesure de la tension efficace de sortie via le multimètre,
5. calcul et stockage du gain en décibels.

Le gain est calculé à partir de la valeur efficace mesurée et de la tension d'entrée RMS, selon :

$$G_{dB}(f_i) = 20 \log_{10} \left(\frac{V_{out,rms}(f_i)}{V_{in,rms}} \right)$$

L'extrait de code suivant illustre la génération de la fréquence logarithmique, la mesure de la tension et le calcul du gain :

Listing 1 – Balayage fréquentiel et calcul du gain

```
1 double f = fLow * pow(fHigh / fLow, (double)i / (nStepsInt - 1));
2 viPrintf(gbf, "FREQ %.6f\n", f);
3 viQueryf(dmm, "MEAS:VOLT:AC?\n", "%lf", &vOut);
4 gainArray[i] = 20.0 * log10(vOut / Vin_rms);
```

Les valeurs de fréquence et de gain sont stockées dans des tableaux afin d'être exploitées ultérieurement pour l'affichage et l'analyse.

2.3 Détermination automatique de la fréquence de coupure

Une fois l'ensemble des points acquis, le programme détermine automatiquement la fréquence de coupure du filtre. L'algorithme mis en œuvre suit les étapes suivantes :

1. recherche du gain maximal $G_{\max}$ sur l'ensemble des points mesurés,
2. calcul du seuil de coupure à -3 dB,
3. identification de la première fréquence pour laquelle le gain devient inférieur ou égal à ce seuil.

La tension efficace de sortie à la fréquence de coupure est ensuite estimée à partir du gain correspondant :

$$V_{out,rms}(f_c) = V_{in,rms} \cdot 10^{\frac{G(f_c)}{20}}$$

L'extrait de code suivant illustre cette procédure :

Listing 2 – Calcul de la fréquence de coupure et de la tension associée

```
1 double gainMax = gainArray[0];
2 for (int i = 1; i < nValid; ++i)
3     if (gainArray[i] > gainMax) gainMax = gainArray[i];
```

```
4
5 double seuil = gainMax - 3.0;
6
7 double fc = freqArray[nValid-1];
8 double gout_fc = gainArray[nValid-1];
9
10 for (int i = 0; i < nValid; ++i)
11 {
12     if (gainArray[i] <= seuil)
13     {
14         fc = freqArray[i];
15         gout_fc = gainArray[i];
16         break;
17     }
18 }
19
20 double vout_fc_rms = Vin_rms * pow(10.0, gout_fc / 20.0);
```

Les valeurs calculées (f_c , gain à f_c et $V_{out,rms}(f_c)$) sont ensuite affichées dans l'interface utilisateur et utilisées pour annoter le diagramme de Bode.

3 Résultats et analyse – Instrumentation directe

Les résultats obtenus à l'issue de l'acquisition permettent de valider le bon fonctionnement de l'application ainsi que le comportement fréquentiel du filtre étudié.

Le diagramme de Bode tracé représente l'évolution du gain en décibels en fonction de la fréquence. On observe une zone de bande passante à bas régime, suivie d'une atténuation progressive lorsque la fréquence augmente. La cassure visible sur la courbe correspond à la fréquence de coupure du filtre, caractéristique d'un comportement de type passe-bas.

À partir des données mesurées et du traitement automatique effectué par le programme, les valeurs suivantes ont été obtenues lors d'un essai représentatif :

- fréquence de coupure estimée : $f_c \approx 1291.55$ Hz,
- tension efficace de sortie à la coupure : $V_{out,rms}(f_c) \approx 0.141$ V,
- gain associé proche de -8 dB.

Ces valeurs sont cohérentes avec l'allure attendue pour un filtre analogique du premier ordre, et confirment la validité du calcul automatique de la fréquence de coupure à partir du seuil -3 dB.

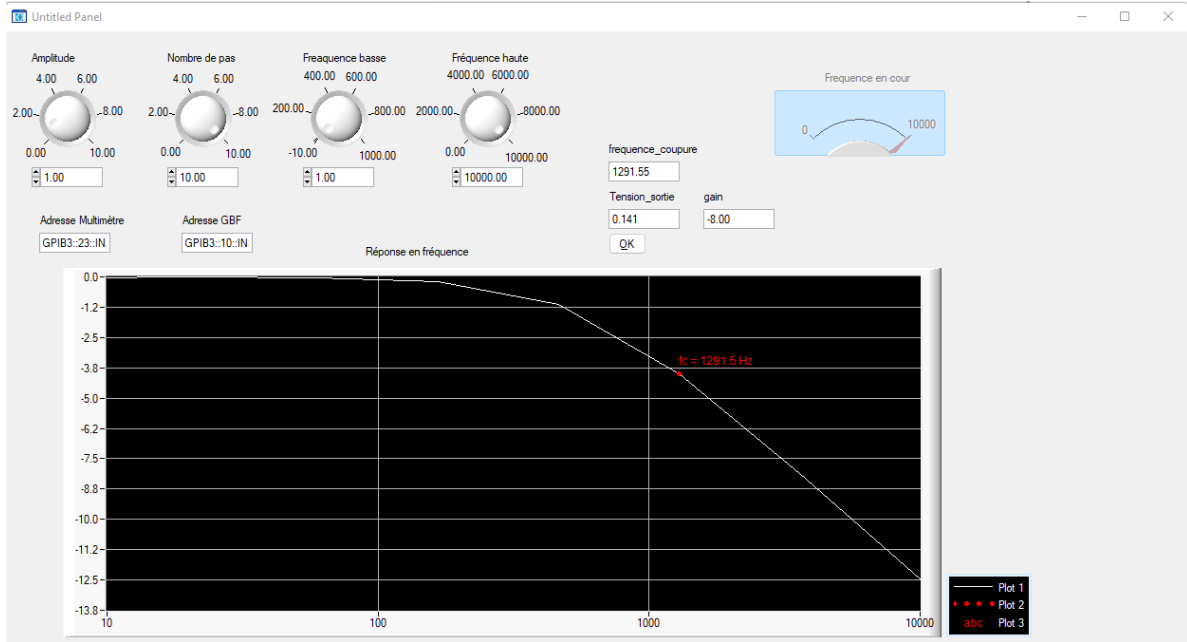


FIGURE 6 – Diagramme de Bode obtenu avec indication automatique de la fréquence de coupure f_c .

3.1 Discussion des résultats

Le filtre étudié est un filtre RC du premier ordre. Sa fréquence de coupure théorique est donnée par la relation classique :

$$f_{c,th} = \frac{1}{2\pi RC}$$

Avec les composants utilisés lors de la manipulation :

$$R = 560 \, \Omega \quad C = 470 \, \text{nF}$$

On obtient :

$$f_{c,th} = \frac{1}{2\pi \times 560 \times 470 \times 10^{-9}} \approx 604 \, \text{Hz}$$

Expérimentalement, l'application a permis d'estimer automatiquement la fréquence de coupure à :

$$f_{c,exp} \approx 1291.55 \, \text{Hz}$$

L'écart observé entre la valeur théorique et la valeur mesurée peut s'expliquer par plusieurs facteurs :

- **Tolérance des composants passifs** : la résistance et la capacité utilisées possèdent des tolérances de fabrication (souvent $\pm 5\%$ ou $\pm 10\%$), ce qui peut entraîner un décalage significatif de la fréquence de coupure réelle.
- **Valeur effective de la capacité** : les condensateurs de type électrolytique ou polyester peuvent présenter une capacité effective différente de la valeur nominale, notamment en fonction de la fréquence.
- **Méthode d'estimation numérique** : la fréquence de coupure est déterminée à partir d'un nombre fini de points. L'absence d'interpolation entre les points du balayage peut conduire à une estimation approximative de f_c .

Malgré cet écart, la forme générale du diagramme de Bode obtenu est cohérente avec celle attendue pour un filtre passe-bas du premier ordre :

- gain quasi constant aux basses fréquences,
- décroissance progressive du gain après la fréquence de coupure,
- pente compatible avec une atténuation de l'ordre de -20 dB/dec.

Les résultats obtenus valident donc le bon fonctionnement de l'application d'instrumentation, tant du point de vue de l'acquisition que du traitement et de l'affichage des données. =====

Deuxième partie

Architecture Client–Serveur TCP/IP

4 Objectifs et architecture générale

4.1 Objectifs de la mise en architecture distribuée

Après la mise en œuvre d'une application d'instrumentation locale (Partie 1), cette seconde partie du TP vise à faire évoluer le système vers une **architecture distribuée Client–Serveur**. L'objectif principal est de dissocier clairement :

- l'**accès au matériel de mesure** (GBF et multimètre),
- l'**interface utilisateur** et le traitement des résultats.

Cette séparation permet :

- de piloter le banc d'instrumentation à distance,
- d'améliorer la modularité du système,
- de préparer une architecture plus réaliste, proche des applications industrielles d'instrumentation réseau.

Les objectifs fonctionnels de cette partie sont donc :

- permettre au client d'envoyer une requête de mesure (amplitude, fréquences, nombre de pas),
- réaliser l'acquisition côté serveur via VISA/GPIB,
- transmettre les résultats au client via TCP/IP,
- assurer côté client le tracé du diagramme de Bode et l'affichage des grandeurs caractéristiques (f_c , gain et tension associée).

4.2 Principe de l'architecture retenue

L'architecture mise en place repose sur un modèle **Client–Serveur TCP/IP** simple et robuste :

- le **serveur** est responsable de :
 - la réception des paramètres de mesure,
 - le pilotage des instruments via VISA,
 - l'acquisition des données expérimentales,
 - l'envoi des résultats au client.
- le **client** assure :
 - la saisie des paramètres par l'utilisateur,
 - la communication TCP avec le serveur,

- la réception et le traitement des données,
- l'affichage graphique et numérique des résultats.

La communication s'effectue via le protocole TCP/IP sur un port dédié (port 5000), garantissant l'intégrité et l'ordre des données échangées.

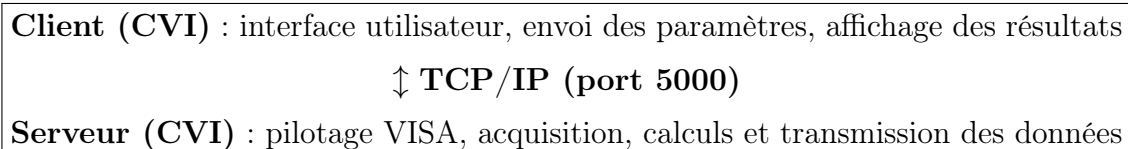


FIGURE 7 – Architecture fonctionnelle Client-Serveur retenue pour l'application

5 Interfaces Client et Serveur

5.1 Interface Serveur : rôle et choix des composants

L'interface du serveur est volontairement **simple et fonctionnelle**. Elle n'est pas destinée à l'analyse des résultats, mais au bon déroulement de l'acquisition.

Elle comporte principalement :

- des champs de saisie pour les **adresses VISA** du GBF et du multimètre,
- un champ indiquant l'**adresse IP du serveur**,
- une zone **Informations** servant de journal d'exécution (logs).

Le choix d'une interface minimaliste permet :

- de réduire les risques d'erreurs de manipulation,
- de se concentrer sur la stabilité de la communication et de l'acquisition,
- de faciliter le débogage grâce aux messages affichés (connexion client, paramètres reçus, état VISA).

5.2 Interface Client : rôle et choix des composants

L'interface client constitue le **poste de pilotage principal** de l'application. Elle regroupe à la fois la configuration de la mesure, le suivi de l'acquisition et l'analyse des résultats.

Elle comprend :

- des **commandes numériques et potentiomètres** pour :
 - l'amplitude du signal,
 - la fréquence minimale et maximale,
 - le nombre de pas du balayage,
- des boutons de **connexion / déconnexion** et de lancement du test,
- une **zone d'informations** affichant les échanges avec le serveur,
- un **graphe de réponse en fréquence** (gain en dB),
- des champs numériques dédiés à l'affichage de :
 - la fréquence de coupure f_c ,
 - le gain associé,
 - la tension efficace de sortie.

Les composants graphiques ont été choisis afin de :

- rendre l'interface intuitive,
- permettre une lecture immédiate des résultats,

- reproduire les instruments de laboratoire classiques (potentiomètres, indicateurs, graphe de Bode).

5.3 Conception et rôle des interfaces Client et Serveur

Dans le cadre de l'architecture Client-Serveur mise en place, deux interfaces distinctes ont été développées sous LabWindows/CVI afin de répondre à des rôles fonctionnels clairement séparés.

Le choix de dissocier l'interface utilisateur finale (client) de l'interface d'accès matériel (serveur) permet :

- d'isoler le pilotage des instruments physiques sur une machine dédiée,
- de simplifier l'interface côté utilisateur,
- de rendre possible un contrôle à distance du banc de mesure via le réseau TCP/IP.

*Interface Serveur L'interface serveur est conçue comme une interface de **supervision et de configuration**. Elle n'a pas vocation à effectuer des calculs ni à afficher des résultats graphiques, mais à assurer le bon déroulement de l'acquisition et de la communication réseau.

Elle regroupe principalement :

- la saisie des **adresses VISA** du multimètre et du GBF (GPIB),
- l'affichage de l'**adresse IP du serveur**, utilisée par le client pour se connecter,
- une **zone d'informations (logs)** permettant de suivre :
 - les connexions et déconnexions du client,
 - la réception des paramètres de mesure,
 - l'état du balayage fréquentiel et des échanges TCP.

Cette interface volontairement épurée facilite le débogage et la supervision, tout en limitant les interactions utilisateur sur la partie critique liée au matériel.

*Interface Client L'interface client constitue l'**interface principale de l'utilisateur**. Elle est responsable du pilotage global de l'expérience, de l'affichage des résultats et de l'analyse des données reçues depuis le serveur.

Elle intègre :

- les réglages des paramètres de mesure :
 - amplitude du signal,
 - fréquences minimale et maximale,
 - nombre de points du balayage,
- les commandes de **connexion / déconnexion** au serveur,
- une **zone d'informations** (chatbox) affichant les messages d'état (connexion, réception des données, fin d'acquisition),
- un **graphe de réponse en fréquence** (diagramme de Bode),
- des champs numériques dédiés à l'affichage des grandeurs caractéristiques :
 - fréquence de coupure f_c ,
 - gain à la fréquence de coupure,
 - tension efficace de sortie associée.

Le choix des composants graphiques (knobs, champs numériques, graphe) permet une interaction intuitive avec l'application tout en offrant une visualisation immédiate des résultats expérimentaux.



5.4 Illustration des interfaces réalisées

Les figures suivantes présentent les interfaces serveur et client développées dans le cadre de l'architecture distribuée.

[t]0.48

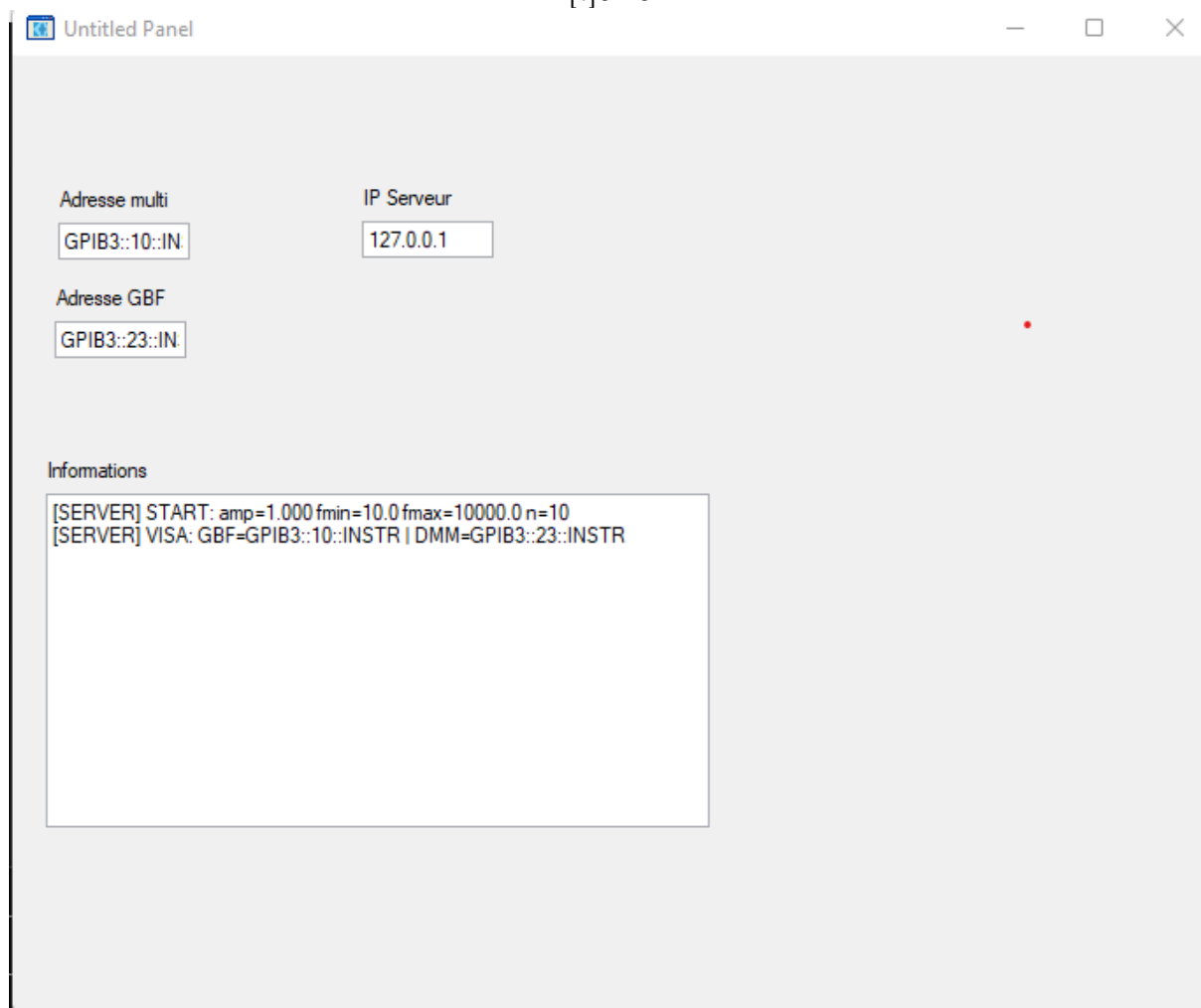


FIGURE 8 – Interface Serveur

[t]0.48

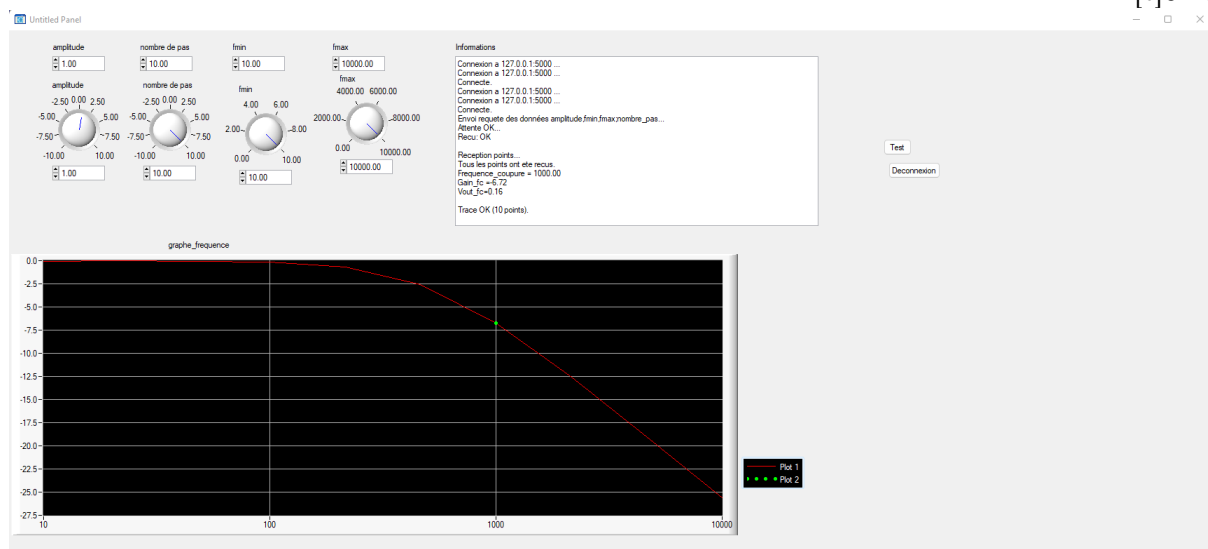


FIGURE 9 – Interface Client

FIGURE 10 – Interfaces Client et Serveur de l'application distribuée

Protocole applicatif TCP

Cette partie décrit le protocole applicatif mis en place au-dessus de la couche TCP/IP afin d'assurer une communication fiable et compréhensible entre le client et le serveur. Le protocole repose sur des échanges textuels simples, lisibles et structurés, facilitant à la fois le débogage et l'évolution future de l'application.

6 Principe général des échanges

La communication suit une logique séquentielle stricte :

- le client transmet les paramètres de mesure,
- le serveur accuse réception,
- le serveur effectue l'acquisition,
- les résultats sont envoyés progressivement,
- la transmission est explicitement terminée.

Ce schéma garantit une synchronisation claire entre les deux entités et évite toute ambiguïté lors de la lecture des données.

7 Requête envoyée par le client

Le client envoie une requête textuelle multi-lignes contenant l'ensemble des paramètres nécessaires à l'acquisition. Chaque paramètre est transmis sous la forme **CLE=VALEUR**, ce qui simplifie le parsing côté serveur. La requête se termine obligatoirement par le mot-clé **START**, signalant le début effectif du traitement.

Listing 3 – Exemple de requête TCP envoyée par le client

```
1 AMP=1.000000
2 FMIN=1.000000
3 FMAX=10000.000000
4 NSTEPS=10
5 START
```

Ce format textuel rend la communication indépendante de l'architecture et facilement extensible (ajout de nouveaux paramètres si nécessaire).

8 Réponse envoyée par le serveur

Après réception et validation des paramètres, le serveur répond selon la séquence suivante :

- envoi de **OK** pour confirmer la prise en compte de la requête,
- transmission des points mesurés sous la forme **frequence;gain**,
- envoi du marqueur **END** pour signaler la fin des données.

Listing 4 – Exemple de réponse TCP côté serveur

```
1 OK
2 10.000000; -0.120000
3 15.848932; -0.180000
4 ...
5 END
```

Ce découpage permet au client de lire les données ligne par ligne sans connaissance préalable du nombre exact de points reçus.

Implémentation côté serveur

Le serveur constitue le cœur du système distribué. Il est responsable à la fois de la communication réseau et de l'accès direct au matériel d'instrumentation via VISA.

9 Cycle de fonctionnement du serveur

Le fonctionnement global du serveur peut être résumé par les étapes suivantes :

1. initialisation du serveur TCP (port 5000),
2. attente d'une connexion cliente,
3. réception et interprétation de la requête de mesure,
4. ouverture et configuration des instruments via VISA,
5. réalisation du balayage fréquentiel,
6. envoi progressif des résultats au client,
7. terminaison de la transmission par END.

Cette organisation séquentielle garantit une exécution déterministe et facilite le suivi de l'état du serveur.

10 Acquisition et envoi des données

Pour chaque fréquence du balayage, le serveur :

- commande le GBF,
- mesure la tension de sortie à l'aide du multimètre,
- calcule le gain correspondant,
- transmet immédiatement le couple `frequence;gain` au client.

Listing 5 – Envoi d'un point de mesure côté serveur

```
1 snprintf(line, sizeof(line), "%.6f;%.6f\n", f, gainDB);  
2 ServerTCPWrite(tcpHandle, line, (unsigned)strlen(line), 2000);
```

Ce choix d'envoi progressif permet au client de commencer le traitement des données sans attendre la fin complète du balayage.

11 Journalisation et supervision côté serveur

Afin de faciliter le suivi de l'exécution et le débogage, le serveur intègre une zone de journalisation affichant les événements importants :

- réception de la commande **START**,
- valeurs des paramètres de mesure,
- ressources VISA effectivement utilisées,
- état général de l'acquisition.

Ces informations permettent de vérifier rapidement la cohérence entre les paramètres envoyés par le client et la configuration réelle du banc de mesure.

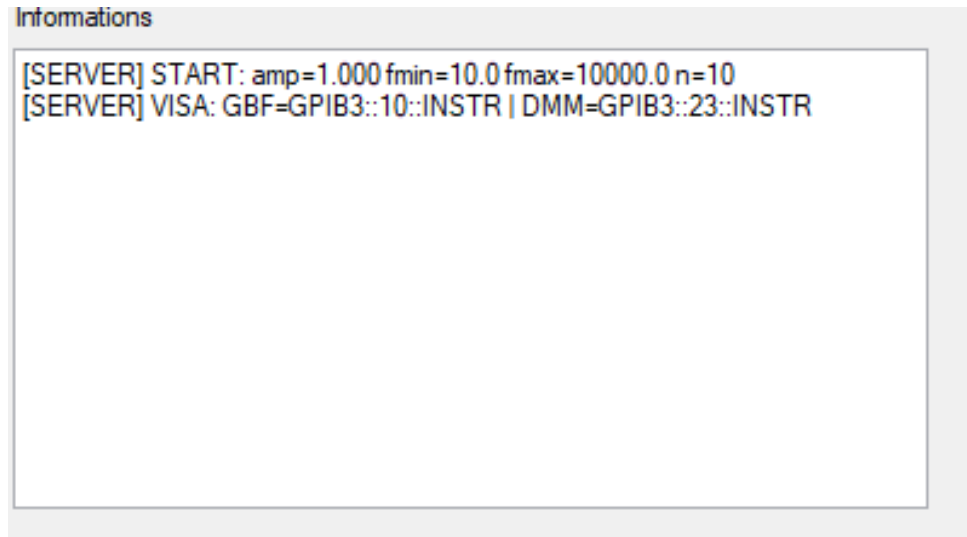


FIGURE 11 – Journalisation côté serveur

12 Implémentation côté client

Le client constitue l'interface utilisateur de l'architecture distribuée. Il est responsable de la saisie des paramètres de mesure, de la communication TCP avec le serveur, de la réception des données et de leur exploitation graphique et numérique.

Contrairement au serveur, le client n'interagit pas directement avec le matériel ; il se concentre exclusivement sur la logique réseau et l'affichage des résultats.

12.1 Gestion de la connexion TCP

Le client établit une connexion TCP vers le serveur (adresse IP et port prédéfinis). L'état de la connexion est immédiatement reflété dans l'interface graphique :

- bouton *Connexion / Déconnexion* mis à jour dynamiquement,
- affichage d'un message de confirmation dans la zone d'informations,
- prévention des actions de test tant que la connexion n'est pas établie.

Cette gestion explicite de l'état réseau évite toute incohérence entre l'interface et la communication réelle.

12.2 Échanges réseau et journalisation côté client

Une fois connecté, le client :

- envoie la requête de mesure au serveur,
- attend explicitement la réponse OK,
- lit ensuite les données ligne par ligne jusqu'au marqueur END.

Chaque étape est consignée dans une zone de journalisation (logs), ce qui permet de suivre précisément le déroulement du protocole et de diagnostiquer rapidement d'éventuels problèmes de communication.

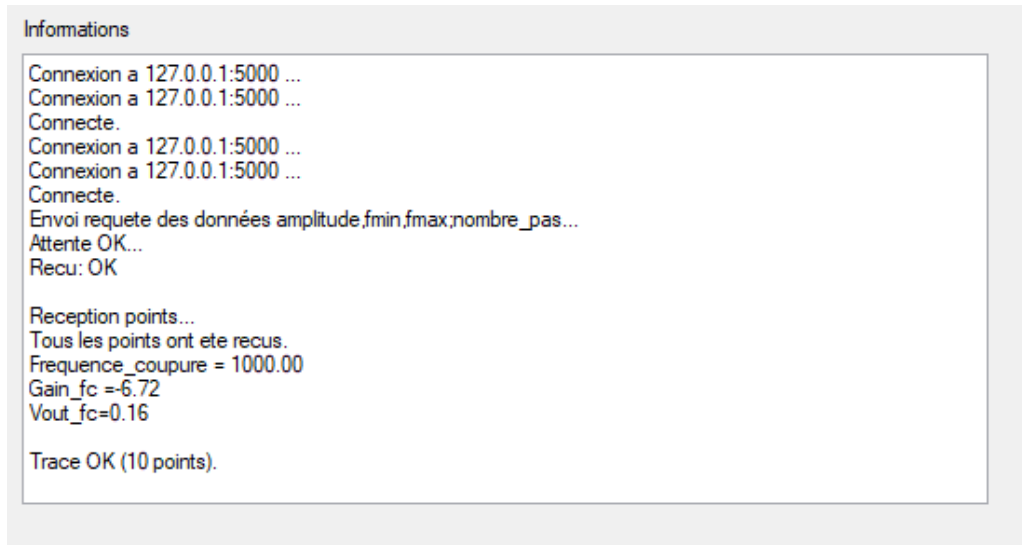


FIGURE 12 – Journalisation côté client

12.3 Réception des données et traitement

Les données reçues sous la forme `frequence;gain` sont :

- parsées ligne par ligne,
- stockées dans des tableaux dynamiques,
- utilisées pour tracer la réponse fréquentielle via `PlotXY`.

Une fois l'ensemble des points reçus, le client applique localement le même algorithme que dans la partie 1 afin de déterminer :

- la fréquence de coupure f_c ,
- le gain associé $G(f_c)$,
- la tension efficace de sortie $V_{out}(f_c)$.

Ce choix de calcul côté client permet de conserver un protocole simple et d'éviter de surcharger le serveur avec des traitements supplémentaires.

12.4 Calcul de la fréquence de coupure côté client

La fréquence de coupure est obtenue par recherche du premier point pour lequel le gain passe sous le seuil -3 dB par rapport au gain maximal :

Listing 6 – Calcul de la fréquence de coupure côté client

```
1 double gainMax = gains[0];
2 for (int i = 1; i < count; ++i)
3     if (gains[i] > gainMax) gainMax = gains[i];
4
5 double seuil = gainMax - 3.0;
6
7 double fc = freqs[count-1];
8 double gout_fc = gains[count-1];
9
10 for (int i = 0; i < count; ++i)
11     if (gains[i] <= seuil) { fc = freqs[i]; gout_fc = gains[i];
        break; }
```

12.5 Affichage des résultats

Les résultats finaux sont affichés directement dans l'interface client :

- tracé du diagramme de Bode (gain en dB),
- affichage numérique de f_c ,
- affichage du gain et de la tension efficace à la fréquence de coupure.

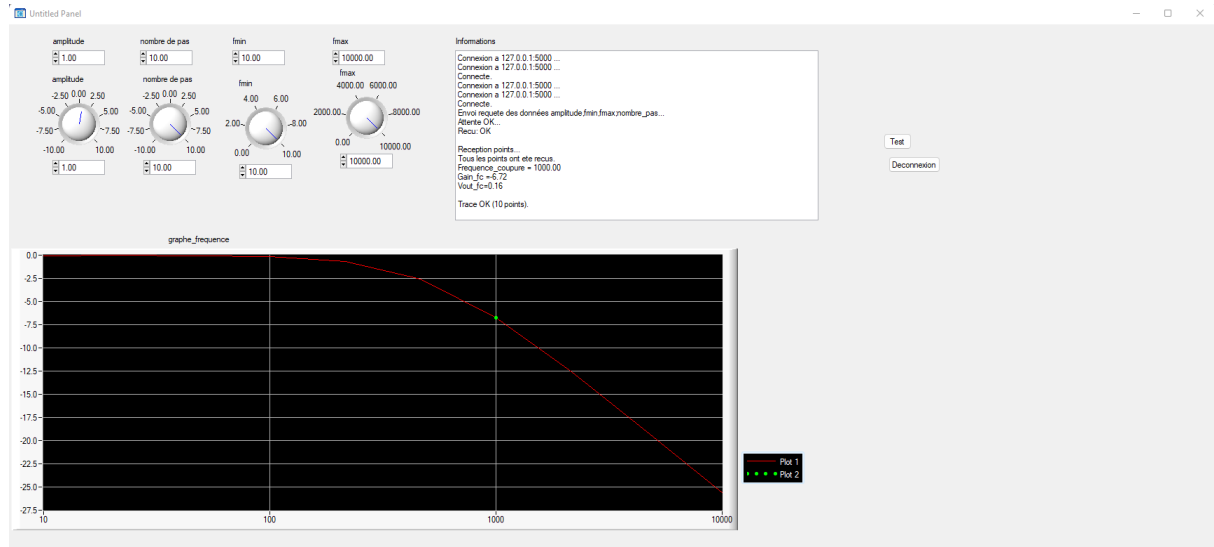


FIGURE 13 – Interface client et affichage des résultats

13 Bilan et améliorations

13.1 Bilan

Ce projet a permis de valider l'ensemble des objectifs fixés, tant sur le plan de l'instrumentation que sur celui du développement logiciel.

Dans un premier temps, la mise en œuvre d'une application d'instrumentation virtuelle en pilotage direct a permis de :

- contrôler un GBF et un multimètre via l'interface VISA/GPIB,
- automatiser l'acquisition de la réponse fréquentielle d'un filtre analogique,
- tracer un diagramme de Bode et déterminer automatiquement la fréquence de coupure.

Dans un second temps, l'extension vers une architecture Client–Serveur a apporté une réelle valeur ajoutée :

- séparation claire des rôles entre acquisition matérielle (serveur) et interface utilisateur (client),
- possibilité de piloter le banc de mesure à distance via TCP/IP,
- meilleure modularité et évolutivité de l'application.

Les résultats obtenus (courbes, fréquence de coupure, gain et tension associée) sont cohérents avec le comportement théorique attendu du filtre et confirment la validité de l'architecture mise en place.

13.2 Difficultés rencontrées

Plusieurs difficultés techniques ont été rencontrées au cours du développement :

- la gestion des **timeouts TCP** et la lecture robuste des messages ligne par ligne (OK, données, END),
- la mise en place d'une **zone de journalisation** (TextBox) avec défilement automatique et gestion correcte des tailles de buffers,
- le débogage des attributs d'interface CVI (attributs non déclarés, limites de taille),
- la synchronisation stricte entre l'envoi de la réponse OK et la transmission des points côté serveur.

Ces difficultés ont toutefois permis d'approfondir la compréhension des mécanismes de communication TCP et de gestion d'interface sous LabWindows/CVI.

13.3 Améliorations possibles

Plusieurs pistes d'amélioration peuvent être envisagées :

- envoyer directement un message structuré de résultats depuis le serveur (RESULT;fc;gain;vout) afin de limiter les calculs côté client,
- améliorer la précision de la fréquence de coupure par interpolation entre les points encadrant le seuil -3 dB,
- gérer plusieurs clients simultanément via une file d'attente et un mécanisme de verrouillage de l'accès VISA,
- structurer davantage les logs avec horodatage et niveaux de gravité (INFO, WARNING, ERROR).

Conclusion générale

Ce TP a permis de concevoir une application complète d'instrumentation virtuelle, depuis le pilotage direct d'instruments de laboratoire jusqu'à une architecture distribuée Client-Serveur basée sur TCP/IP. L'automatisation de l'acquisition, le traitement des données et l'affichage des résultats ont conduit à une caractérisation fiable de la réponse fréquentielle d'un filtre analogique.

Au-delà des résultats obtenus, ce travail a permis de consolider des compétences essentielles en instrumentation, communication réseau et conception logicielle, tout en mettant en évidence l'intérêt des architectures distribuées dans les systèmes de mesure modernes.

=====

Annexes

A Annexe A – Code source (Partie 1)

A.1 Fichier TP1C.txt (complet)

Listing 7 – TP1C.txt

```
1 #include <cvirte.h>
2 #include <userint.h>
3 #include "Intruvirtuelle.h"
4 #include <ansi_c.h>
5 #include <utility.h>
```

```
6  #include <stdlib.h>
7  #include <math.h>
8  #include <visa.h>
9
10 /* Sessions VISA globales */
11 static ViSession rm = VI_NULL;
12 static ViSession dmm = VI_NULL;      /* multimetre */
13 static ViSession gbf = VI_NULL;     /* generateur 33220A */
14
15 static int panelHandle = 0;
16
17 /* ----- Prototypes internes ----- */
18 static int OpenVisaInstruments(const char *addrDmm, const char
    *addrGbf);
19 static void CloseVisaInstruments(void);
20
21 /*
    =====
    */
22 /*                                main()
    */
23 /*
    =====
    */
24
25 int main (int argc, char *argv[])
26 {
27     if (InitCVIRTE (0, argv, 0) == 0)
28         return -1;      /* out of memory */
29
30     panelHandle = LoadPanel (0, "Intruvirtuelle.uir", PANEL);
31     if (panelHandle < 0)
32         return -1;
33
34     DisplayPanel (panelHandle);
35     RunUserInterface ();
36     DiscardPanel (panelHandle);
37
38     return 0;
39 }
40
41 /*
    =====
    */
42 /*                                Callback du panneau
    */
43 /*
    =====
    */
44
```

```
45 int CVICALLBACK cbPanel(int panel, int event, void
    *callbackData,
46                               int eventData1, int eventData2)
47 {
48     switch (event)
49     {
50         case EVENT_CLOSE:
51             QuitUserInterface (0);
52             break;
53     }
54     return 0;
55 }
56
57 /*
=====
58 */
59 /*          Callback de lancement du sweep
          */
60 /*          attacher      un bouton "Lancer" ou "OK" dans le .uir
          */
61 /*
=====
62 */
63
64 int CVICALLBACK cbSweep(int panel, int control, int event,
    void *callbackData, int eventData1,
    int eventData2)
65 {
66     if (event != EVENT_COMMIT)
67         return 0;
68
69     double amplitude = 0.0;
70     double fLow = 0.0, fHigh = 0.0;
71     double nSteps = 0;
72     char addrDmm[256] = {0};
73     char addrGbf[256] = {0};
74
75     GetCtrlVal(panel, PANEL_NUMERICKNOB, &amplitude);
76     // Amplitude
77     GetCtrlVal(panel, PANEL_NUMERICKNOB_2, &nSteps);
78     // Nombre de pas
79     GetCtrlVal(panel, PANEL_NUMERICKNOB_3, &fLow);
80     // Fr quence basse
81     GetCtrlVal(panel, PANEL_NUMERICKNOB_4, &fHigh);
82     // Fr quence haute
83     GetCtrlVal(panel, PANEL_STRING, addrDmm);
84     // Adresse Multim tre
85     GetCtrlVal(panel, PANEL_STRING_2, addrGbf);
86     // Adresse GBF
87
88     if (nSteps < 2 || fHigh <= fLow || amplitude <= 0.0)
```

```
82     {
83         MessagePopup("Erreur param tres",
84                     "V rifiez: amplitude > 0, nSteps >= 2 et
                        f_high > f_low.");
85         return 0;
86     }
87
88     /* 2. Ouvrir les instruments VISA */
89     if (OpenVisaInstruments(addrDmm, addrGbf) != 0)
90     {
91         MessagePopup("Erreur", "Impossible d'ouvrir les
                        instruments VISA.");
92         return 0;
93     }
94
95
96     double Vin_rms = amplitude / (2.0 * sqrt(2.0));
97     if (Vin_rms <= 0.0)
98         Vin_rms = 1e-9;
99
100
101     DeleteGraphPlot(panel, PANEL_GRAPH, -1,
102                     VAL_IMMEDIATE_DRAW);
103
104
105     int nStepsInt = (int)nSteps;
106     double *freqArray = (double*)malloc(nStepsInt *
107                                         sizeof(double));
108     double *gainArray = (double*)malloc(nStepsInt *
109                                         sizeof(double));
110
111     if (!freqArray || !gainArray)
112     {
113         MessagePopup("Erreur", "M moire insuffisante.");
114         CloseVisaInstruments();
115         if (freqArray) free(freqArray);
116         if (gainArray) free(gainArray);
117         return 0;
118     }
119
120     /* 4. Configuration initiale du GBF */
121     viPrintf(gbf, "OUTP:LOAD INF\n");           // Haute
122         imp dance
123     viPrintf(gbf, "FUNC SIN\n");                 // Forme
124         sinuso dale
125     viPrintf(gbf, "VOLT:OFFS 0\n");              // Offset      OV
126     viPrintf(gbf, "OUTP ON\n");                  // Activer la
127         sortie
128
129     /* 5. Boucle de sweep en fr quence */
130
131
```

```
125     int nValid = 0;
126
127     for (int i = 0; i < nStepsInt; ++i)
128     {
129
130         double f = fLow*pow(fHigh/fLow,(double)i/(nStepsInt -
131             1));
132         double vOut = 0.0;
133         ViStatus status;
134
135         freqArray[i] = f;
136
137         status = viPrintf(gbf, "FREQ %.6f\n", f);
138         if (status < VI_SUCCESS)
139         {
140             char msg[256];
141             sprintf(msg, "Erreur GBF FREQ      f=%.1f Hz :
142                 0x%08X", f, (unsigned int)status);
143             MessagePopup("Erreur VISA", msg);
144             break;
145         }
146
147         status = viPrintf(gbf, "VOLT %.1f\n", amplitude);
148         if (status < VI_SUCCESS)
149         {
150             char msg[256];
151             sprintf(msg, "Erreur GBF VOLT      f=%.1f Hz :
152                 0x%08X", f, (unsigned int)status);
153             MessagePopup("Erreur VISA", msg);
154             break;
155         }
156
157         SetCtrlVal(panel, PANEL_RINGMETER, (int)f);
158         ProcessSystemEvents();
159
160         Delay(0.2);
161
162         status = viQueryf(dmm, "MEAS:VOLT:AC?\n", "%lf",
163             &vOut);
164         if (status < VI_SUCCESS)
165         {
166             char msg[256];
167             sprintf(msg, "Erreur mesure DMM      f=%.1f Hz :
168                 0x%08X", f, (unsigned int)status);
169             MessagePopup("Erreur VISA", msg);
170             break;
171         }
172
173         if (vOut > 1e-9 && Vin_rms > 1e-9)
174         {
175             gainArray[i] = 20.0 * log10(vOut / Vin_rms);
176         }
177     }
```

```
171     }
172
173     nValid++;
174     ProcessSystemEvents();
175 }
176
177 viPrintf(gbf, "OUTP OFF\n");
178
179 if (nValid > 0)
180 {
181
182     double gainMax = gainArray[0];
183     for (int i = 1; i < nValid; ++i)
184         if (gainArray[i] > gainMax) gainMax = gainArray[i];
185
186     double seuil = gainMax - 3.0;
187
188     double fc = freqArray[nValid-1];
189     double gout_fc = gainArray[nValid-1];
190
191     for (int i = 0; i < nValid; ++i)
192     {
193         if (gainArray[i] <= seuil)
194         {
195             fc = freqArray[i];
196             gout_fc = gainArray[i];
197             break;
198         }
199     }
200
201
202
203     /* 8. Tension de sortie      fc (RMS) */
204     double vout_fc_rms = Vin_rms * pow(10.0, gout_fc /
205         20.0);
206
207     /* Affichage des r sultats */
208     char buf[64];
209
210     sprintf(buf, "%.2f", fc);
211     SetCtrlVal(panel, PANEL_STRING_3, buf);           //
212         Fr quence de coupure
213
214     sprintf(buf, "%.2f", gout_fc);
215     SetCtrlVal(panel, PANEL_STRING_4, buf);           //
216         Gain      fc
217
218     sprintf(buf, "%.3f", vout_fc_rms);
219     SetCtrlVal(panel, PANEL_STRING_5, buf);           //
220         Tension sortie      fc
221
222 }
```

```
218      /*Tracer la courbe G(f) sur le graphe avec marqueur de
        fc */
219      PlotXY(panel, PANEL_GRAPH,
220             freqArray, gainArray,
221             nValid,
222             VAL_DOUBLE, VAL_DOUBLE,
223             VAL_THIN_LINE, VAL_EMPTY_SQUARE,
224             VAL_SOLID, 1, VAL_WHITE);
225
226      /* Ajouter un marqueur rouge la fr quence de
        coupure */
227      double fc_marker[1];
228      double gain_marker[1];
229      fc_marker[0] = fc;
230      gain_marker[0] = gout_fc;
231
232      PlotXY(panel, PANEL_GRAPH,
233             fc_marker, gain_marker,
234             1,
235             VAL_DOUBLE, VAL_DOUBLE,
236             VAL_SCATTER, VAL_SOLID_CIRCLE,
237             VAL_SOLID, 10, VAL_RED); /* Point rouge,
        taille 10 */
238
239      /* Ajouter une annotation textuelle "fc = XXX Hz" */
240      char annotation[64];
241      sprintf(annotation, "fc = %.1f Hz", fc);
242      PlotText(panel, PANEL_GRAPH, fc, gout_fc + 0.5,
243              annotation,
244              VAL_APP_META_FONT, VAL_RED, VAL_TRANSPARENT);
245      }
246      else if (nValid > 0)
247      {
248          /* Si pas de calcul de fc mais des donn es valides,
        tracer quand m me */
249          PlotXY(panel, PANEL_GRAPH,
250                 freqArray, gainArray,
251                 nValid,
252                 VAL_DOUBLE, VAL_DOUBLE,
253                 VAL_THIN_LINE, VAL_EMPTY_SQUARE,
254                 VAL_SOLID, 1, VAL_WHITE);
255      }
256
257      /* Lib ration de la m moire */
258      free(freqArray);
259      free(gainArray);
260
261      /* 10. Fermer les instruments VISA */
262      CloseVisaInstruments();
```

```
263     MessagePopup("Sweep termin ", "L'acquisition de la  
        r p nse en fr quence est termin e.");  
264  
265     return 0;  
266 }  
267  
268 /*  
=====   
*/  
269 /*              Fonctions internes d'ouverture VISA  
        */  
270 /*  
=====   
*/  
271  
272 static int OpenVisaInstruments(const char *addrDmm, const char  
        *addrGbf)  
273 {  
274     ViStatus status;  
275  
276     /* Ouvrir le Resource Manager */  
277     if (viOpenDefaultRM(&rm) < VI_SUCCESS)  
278     {  
279         MessagePopup("Erreur VISA", "Impossible d'ouvrir le  
            Resource Manager VISA.");  
280         return -1;  
281     }  
282  
283     /* Ouvrir le GBF (g n rateur) */  
284     status = viOpen(rm, (ViRsrc)addrGbf, VI_NULL, VI_NULL,  
        &gbf);  
285     if (status < VI_SUCCESS)  
286     {  
287         char msg[512];  
288         sprintf(msg, "Impossible d'ouvrir le GBF  
            l'adresse:\n%s\n\nCode erreur: 0x%08X\n\nV rifiez  
            l'adresse et la connexion.",  
289             addrGbf, (unsigned int)status);  
290         MessagePopup("Erreur VISA GBF", msg);  
291         viClose(rm);  
292         rm = VI_NULL;  
293         return -1;  
294     }  
295  
296     /* Ouvrir le multim tre */  
297     status = viOpen(rm, (ViRsrc)addrDmm, VI_NULL, VI_NULL,  
        &dmm);  
298     if (status < VI_SUCCESS)  
299     {  
300         char msg[512];
```

```
301     sprintf(msg, "Impossible d'ouvrir le multimetre
        l'adresse:\n%s\n\nCode erreur: 0x%08X\n\nVérifiez
        l'adresse et la connexion.",
302         addrDmm, (unsigned int)status);
303     MessagePopup("Erreur VISA Multimetre", msg);
304     viClose(gbf);
305     viClose(rm);
306     gbf = VI_NULL;
307     rm = VI_NULL;
308     return -1;
309 }
310
311 /* Configuration des timeouts (5 secondes) */
312 viSetAttribute(gbf, VI_ATTR_TMO_VALUE, 5000);
313 viSetAttribute(dmm, VI_ATTR_TMO_VALUE, 5000);
314
315 /* Test de communication avec les instruments */
316 char idn[256];
317
318 /* Test GBF */
319 status = viQueryf(gbf, "*IDN?\n", "%t", idn);
320 if (status < VI_SUCCESS)
321 {
322     MessagePopup("Avertissement", "Le GBF ne répond pas
        la commande *IDN?");
323 }
324
325 /* Test Multimetre */
326 status = viQueryf(dmm, "*IDN?\n", "%t", idn);
327 if (status < VI_SUCCESS)
328 {
329     MessagePopup("Avertissement", "Le multimetre ne
        répond pas la commande *IDN?");
330 }
331
332 return 0;
333 }
334
335 static void CloseVisaInstruments(void)
336 {
337     /* Désactiver la sortie du GBF avant de fermer */
338     if (gbf != VI_NULL)
339     {
340         viPrintf(gbf, "OUTP OFF\n");
341     }
342
343     /* Fermer les sessions dans l'ordre inverse */
344     if (dmm != VI_NULL)
345     {
346         viClose(dmm);
347         dmm = VI_NULL;
```

```
348     }
349
350     if (gbf != VI_NULL)
351     {
352         viClose(gbf);
353         gbf = VI_NULL;
354     }
355
356     if (rm != VI_NULL)
357     {
358         viClose(rm);
359         rm = VI_NULL;
360     }
361 }*
```

B Annexe B – Code source (Serveur TCP/VISA)

B.1 Fichier ServeurC.txt (complet)

Listing 8 – ServeurC.txt

```
1  #include <cvirte.h>
2  #include <userint.h>
3  #include <ansi_c.h>
4  #include <utility.h>
5  #include <math.h>
6  #include <string.h>
7
8  #include <tcpsupp.h>
9  #include <visa.h>
10
11 #include "serveur.h"
12
13 #define SERVER_PORT 5000
14
15 /* ===== Globals ===== */
16 static int panelHandle = 0;
17 static int g_serverRunning = 0;
18
19 static ViSession rm = VI_NULL;
20 static ViSession dmm = VI_NULL;
21 static ViSession gbf = VI_NULL;
22
23
24
25 static void CloseVisa(void)
26 {
27     if (gbf != VI_NULL) { viPrintf(gbf, "OUTP OFF\n");
28                         viClose(gbf); gbf = VI_NULL; }
29     if (dmm != VI_NULL) { viClose(dmm); dmm = VI_NULL; }
```

```
29     if (rm != VI_NULL) { viClose(rm); rm = VI_NULL; }
30 }
31
32 static int OpenVisa(const char *addrDmm, const char *addrGbf)
33 {
34     if (viOpenDefaultRM(&rm) < VI_SUCCESS) return -1;
35     if (viOpen(rm, (ViRsrc)addrGbf, VI_NULL, VI_NULL, &gbf) <
        VI_SUCCESS) { CloseVisa(); return -2; }
36     if (viOpen(rm, (ViRsrc)addrDmm, VI_NULL, VI_NULL, &dmm) <
        VI_SUCCESS) { CloseVisa(); return -3; }
37
38     viSetAttribute(gbf, VI_ATTR_TMO_VALUE, 5000);
39     viSetAttribute(dmm, VI_ATTR_TMO_VALUE, 5000);
40     return 0;
41 }
42
43 /* ===== Sweep + envoi TCP ===== */
44 static void RunSweepAndSend(unsigned tcpHandle,
45                             const char *addrDmm, const char
46                             *addrGbf,
47                             double ampVpp, double fmin, double
48                             fmax, int nSteps)
49 {
50     if (ampVpp <= 0.0) ampVpp = 1.0;
51     if (fmin <= 0.0) fmin = 10.0;
52     if (fmax <= fmin) fmax = 10000.0;
53     if (nSteps < 2) nSteps = 50;
54
55     double Vin_rms = ampVpp / (2.0 * sqrt(2.0));
56     if (Vin_rms < 1e-12) Vin_rms = 1e-12;
57
58     double *freqArray = (double*)malloc(nSteps *
59                                         sizeof(double));
60     double *gainArray = (double*)malloc(nSteps *
61                                         sizeof(double));
62     if (OpenVisa(addrDmm, addrGbf) != 0)
63     {
64         ServerTCPWrite(tcpHandle, "ERR:VISA\n", 9, 2000);
65         return;
66     }
67
68     /* Config GBF */
69     viPrintf(gbf, "OUTP:LOAD INF\n");
70     viPrintf(gbf, "FUNC SIN\n");
71     viPrintf(gbf, "VOLT:OFFS 0\n");
72     viPrintf(gbf, "VOLT %.6f\n", ampVpp);
73     viPrintf(gbf, "OUTP ON\n");
74
75     ServerTCPWrite(tcpHandle, "OK\n", 3, 2000);
76
77     for (int i = 0; i < nSteps; ++i)
```

```
74     {
75         double ratio = (double)i / (double)(nSteps - 1);
76         double f = fmin * pow(fmax / fmin, ratio);
77         freqArray[i]=f;
78
79         if (viPrintf(gbf, "FREQ %.6f\n", f) < VI_SUCCESS)
80             break;
81
82         Delay(0.15);
83
84         double vOut_rms = 0.0;
85         ViStatus st = viQueryf(dmm, "MEAS:VOLT:AC?\n", "%lf",
86             &vOut_rms);
87         if (st < VI_SUCCESS || vOut_rms < 1e-12) vOut_rms =
88             1e-12;
89
90         double gainDB = 20.0 * log10(vOut_rms / Vin_rms);
91         gainArray[i]=gainDB;
92
93         char line[128];
94         snprintf(line, sizeof(line), "%.6f;%.6f\n", f, gainDB);
95         ServerTCPWrite(tcpHandle, line,
96             (unsigned)strlen(line), 2000);
97         ProcessSystemEvents();
98     }
99
100     ServerTCPWrite(tcpHandle, "END\n", 4, 2000);
101     CloseVisa();
102 }
103
104 /* ===== TCP callback ===== */
105 static int CVICALLBACK TcpServerCB(unsigned handle, int xType,
106     int errCode, void *callbackData)
107 {
108     if (xType == TCP_CONNECT) {
109         SetCtrlVal(panelHandle, PANEL_STRING_3, "127.0.0.1");
110         printf("[SERVER] Client connecte.\n"); return 0; }
111     if (xType == TCP_DISCONNECT) { printf("[SERVER] Client
112         d connecte.\n"); return 0; }
113
114     if (xType == TCP_DATAREADY)
115     {
116         char req[2048];
117         int r = ServerTCPRead(handle, req, sizeof(req)-1,
118             3000);
119         if (r <= 0) return 0;
120         req[r] = '\0';
121
122         /* Lire adresses depuis UI */
123         char addrDmm[256] = {0};
```

```
118     char addrGbf[256] = {0};
119     GetCtrlVal(panelHandle, PANEL_STRING, addrDmm);
120     GetCtrlVal(panelHandle, PANEL_STRING_2, addrGbf);
121
122     /* Param tres par d faut + parsing */
123     double amp = 1.0, fmin = 10.0, fmax = 10000.0;
124     int nSteps = 50;
125
126     char *p;
127     p = strstr(req, "AMP="); if (p) amp = atof(p + 4);
128     p = strstr(req, "FMIN="); if (p) fmin = atof(p + 5);
129     p = strstr(req, "FMAX="); if (p) fmax = atof(p + 5);
130     p = strstr(req, "NSTEPS="); if (p) nSteps = atoi(p + 7);
131
132     char string[100];
133     char string2[100];
134
135     snprintf(string, sizeof(string),
136              "[SERVER] START: amp=%.3f fmin=%.1f fmax=%.1f\n",
137              amp, fmin, fmax, nSteps);
138
139     snprintf(string2, sizeof(string2),
140              "[SERVER] VISA: GBF=%s | DMM=%s\n",
141              addrGbf, addrDmm);
142
143     if (strstr(req, "START"))
144     {
145         SetCtrlVal(panelHandle, PANEL_TEXTBOX, string);
146         SetCtrlVal(panelHandle, PANEL_TEXTBOX, string2);
147         RunSweepAndSend(handle, addrDmm, addrGbf, amp,
148                         fmin, fmax, nSteps);
149     }
150     else
151     {
152         ServerTCPWrite(handle, "ERR\n", 4, 2000);
153     }
154     return 0;
155 }
156
157 /* ===== main / panel ===== */
158 int main(int argc, char *argv[])
159 {
160     if (!InitCVIRTE(0, argv, 0)) return -1;
161
162     panelHandle = LoadPanel(0, "serveur.uir", PANEL);
163     if (panelHandle < 0) return -1;
164
165     int err = RegisterTCPServer(SERVER_PORT, TcpServerCB,
166                                NULL);
```

```
166     if (err < 0) MessagePopup("TCP Server",
167                               GetTCPErrString(err));
168     else { g_serverRunning = 1; printf("[SERVER] D m a r r
169           port %d\n", SERVER_PORT); }
170
171     DisplayPanel(panelHandle);
172     RunUserInterface();
173     DiscardPanel(panelHandle);
174     return 0;
175 }
176
177 int CVICALLBACK cbPanel(int panel, int event, void *cb, int
178     e1, int e2)
179 {
180     if (event == EVENT_CLOSE)
181     {
182         if (g_serverRunning) UnregisterTCPServer(SERVER_PORT);
183         CloseVisa();
184         QuitUserInterface(0);
185     }
186     return 0;
187 }
```

C Annexe C – Code source (Client TCP/UI)

C.1 Fichier clientC.txt (complet)

Listing 9 – clientC.txt

```
1  #include <cvirte.h>
2  #include <userint.h>
3  #include <ansi_c.h>
4  #include <utility.h>
5  #include <tcpsupp.h>
6  #include <string.h>
7  #include <stdlib.h>
8  #include <stdarg.h>
9
10 #include "Client.h"
11
12 #define SERVER_PORT 5000
13 #define SERVER_IP "127.0.0.1"
14
15 /* globals */
16 static int panelHandle = 0;
17 static unsigned int g_tcpHandle = 0;
18 static int g_connected = 0;
19
20 /* ===== UI log (chatbox) ===== */
```

```
21 static void LogUI(const char *fmt, ...)
22 {
23     char oldText[14546] = {0};
24     char newLine[2048] = {0};
25     char out[8700] = {0};
26
27     va_list args;
28     va_start(args, fmt);
29     vsnprintf(newLine, sizeof(newLine), fmt, args);
30     va_end(args);
31
32     printf("[CLIENT] %s\n", newLine);
33
34     if (panelHandle > 0)
35     {
36
37         GetCtrlVal(panelHandle, PANEL_TEXTBOX, oldText);
38         snprintf(out, sizeof(out), "%s%s\r\n", oldText,
39                 newLine);
40         SetCtrlVal(panelHandle, PANEL_TEXTBOX, out);
41
42         /* scroll bas */
43         SetCtrlAttribute(panelHandle, PANEL_TEXTBOX,
44                         ATTR_TEXT_SELECTION_START, (int)strlen(out));
45     }
46 }
47
48 /* ===== TCP read line (robuste) ===== */
49 /* Lit 1 ligne terminée par '\n'. Gère le fait que TCP peut
50 renvoyer plusieurs lignes d'un coup. */
51
52 static int TcpReadLine(unsigned int handle, char *out, int
53 outSize)
54 {
55     int total = 0;
56     double t0 = Timer();
57
58     while (total < outSize - 1)
59     {
60         char c = 0;
61         int r = ClientTCPRead(handle, &c, 1, 2000); // timeout
62             2s
63         if (r <= 0) break;
64
65         out[total++] = c;
66         if (c == '\n') break;
67
68         if ((Timer() - t0) > 10.0) break; // s'arrête
69     }
70 }
```

```
66     out[total] = '\\0';
67     return total;
68 }
69
70
71
72 static int TcpSendAll(unsigned int handle, const char *msg)
73 {
74     int len = (int)strlen(msg);
75     int sent = ClientTCPWrite(handle, msg, len, 3000);
76     return (sent < 0) ? -1 : 0;
77 }
78
79 /* ===== main ===== */
80 int main (int argc, char *argv[])
81 {
82     if (!InitCVIRTE(0, argv, 0))
83         return -1;
84
85     panelHandle = LoadPanel(0, "client.uir", PANEL);
86     if (panelHandle < 0)
87         return -1;
88
89     /* vider chatbox au d marriage */
90     SetCtrlVal(panelHandle, PANEL_TEXTBOX, "");
91
92     DisplayPanel(panelHandle);
93     RunUserInterface();
94     DiscardPanel(panelHandle);
95     return 0;
96 }
97
98 int CVICALLBACK cbPanel(int panel, int event, void
99 *callbackData, int eventData1, int eventData2)
100 {
101     if (event == EVENT_CLOSE)
102     {
103         if (g_connected && g_tcpHandle != 0)
104         {
105             DisconnectFromTCPServer(g_tcpHandle);
106             g_tcpHandle = 0;
107             g_connected = 0;
108         }
109         QuitUserInterface(0);
110     }
111     return 0;
112 }
113
114 /* ===== Connexion ===== */
115 int CVICALLBACK cbConnexion(int panel, int control, int event,
```

```
115 void *callbackData, int
      eventData1, int eventData2)
116 {
117     if (event != EVENT_COMMIT) return 0;
118
119     if (!g_connected)
120     {
121         LogUI("Connexion a %s:%d ...", SERVER_IP, SERVER_PORT);
122
123         int err = ConnectToTCPServer(&g_tcpHandle,
            SERVER_PORT, SERVER_IP, NULL, NULL, 5000);
124         if (err < 0)
125         {
126             MessagePopup("Connexion", GetTCPErrorString(err));
127             LogUI("ECHEC connexion: %s",
                GetTCPErrorString(err));
128             g_tcpHandle = 0;
129             g_connected = 0;
130             return 0;
131         }
132
133         g_connected = 1;
134         SetCtrlAttribute(panel, PANEL_COMMANDBUTTON_CONN,
            ATTR_LABEL_TEXT, "Deconnexion");
135         LogUI("Connecte.");
136     }
137     else
138     {
139         DisconnectFromTCPServer(g_tcpHandle);
140         g_tcpHandle = 0;
141         g_connected = 0;
142         SetCtrlAttribute(panel, PANEL_COMMANDBUTTON_CONN,
            ATTR_LABEL_TEXT, "Connexion");
143         LogUI("Disconnecte.");
144     }
145     return 0;
146 }
147
148 /* ===== Test ===== */
149 int CVICALLBACK cbTest(int panel, int control, int event,
150 void *callbackData, int eventData1, int
      eventData2)
151 {
152     if (event != EVENT_COMMIT) return 0;
153
154     if (!g_connected)
155     {
156         MessagePopup("Erreur", "Connecte-toi d'abord au
            serveur.");
157         return 0;
158     }
```

```
159
160     double amp = 0.0, fmin = 0.0, fmax = 0.0;
161     double nSteps_d = 0.0;
162     int nSteps = 0;
163
164     /* Tes IDs actuels */
165     GetCtrlVal(panel, PANEL_NUMERICDIAL, &amp);
166     GetCtrlVal(panel, PANEL_NUMERICDIAL_3, &fmin);
167     GetCtrlVal(panel, PANEL_NUMERICDIAL_4, &fmax);
168     GetCtrlVal(panel, PANEL_NUMERICDIAL_2, &nSteps_d);
169     nSteps = (int)(nSteps_d + 0.5);
170
171     if (amp <= 0.0 || fmin <= 0.0 || fmax <= fmin || nSteps <
172         2)
173     {
174         MessagePopup("Parametres invalides", "Verifie amp>0,
175             fmin>0, fmax>fmin, nSteps>=2.");
176         return 0;
177     }
178
179     char req[256];
180     snprintf(req, sizeof(req),
181         "AMP=%.6f\nFMIN=%.6f\nFMAX=%.6f\nNSTEPS=%d\nSTART\n",
182         amp, fmin, fmax, nSteps);
183
184     LogUI("Envoi requete des donn es
185         amplitude,fmin,fmax;nombre_pas...");
186     if (TcpSendAll(g_tcpHandle, req) != 0)
187     {
188         MessagePopup("Erreur", "Envoi TCP echoue.");
189         LogUI("Erreur envoi TCP.");
190         return 0;
191     }
192
193     /* Lire OK (timeout 6s) */
194     char line[512];
195     LogUI("Attente OK...");
196     int n = TcpReadLine(g_tcpHandle, line, sizeof(line));
197     if (n <= 0)
198     {
199         MessagePopup("Erreur", "Timeout: pas de reponse du
200             serveur.");
201         LogUI("Timeout: serveur ne repond pas.");
202         return 0;
203     }
204
205     LogUI("Recu: %s", line);
206
207     if (strncmp(line, "OK", 2) != 0)
208     {
209         MessagePopup("Serveur", line);
```

```
206     LogUI("Serveur a repondu erreur (pas OK).");
207     return 0;
208 }
209
210 double *freqs = (double*)malloc(nSteps * sizeof(double));
211 double *gains = (double*)malloc(nSteps * sizeof(double));
212 if (!freqs || !gains)
213 {
214     free(freqs); free(gains);
215     MessagePopup("Erreur", "Memoire insuffisante.");
216     return 0;
217 }
218
219 int count = 0;
220 LogUI("Reception points...");
221
222 while (1)
223 {
224     n = TcpReadLine(g_tcpHandle, line, sizeof(line));
225     if (n <= 0)
226     {
227         LogUI("Timeout pendant reception.");
228         break;
229     }
230
231     if (strncmp(line, "END", 3) == 0)
232     {
233         LogUI("Tous les points ont ete recus.");
234         break;
235     }
236
237     double f = 0.0, g = 0.0;
238
239     if (sscanf(line, "%lf;%lf", &f, &g) == 2)
240     {
241         if (count < nSteps)
242         {
243             freqs[count] = f;
244             gains[count] = g;
245             count++;
246         }
247     }
248 }
249
250
251
252 if (count > 0)
253 {
254
255     double gainMax = gains[0];
256     for (int i = 1; i < count; ++i)
```

```
257         if (gains[i] > gainMax) gainMax = gains[i];
258
259         double seuil = gainMax - 3.0;
260
261         double fc = freqs[count-1];
262         double gout_fc = gains[count-1];
263
264         for (int i = 0; i < count; ++i)
265         {
266             if (gains[i] <= seuil)
267             {
268                 fc = freqs[i];
269                 gout_fc = gains[i];
270                 break;
271             }
272         }
273
274
275         double Vin_rms = amp / (2.0 * sqrt(2.0));
276         if (Vin_rms < 1e-12) Vin_rms = 1e-12;
277
278         double vout_fc_rms = Vin_rms * pow(10.0, gout_fc /
279             20.0);
280
281         /* Affichage des r sultats */
282         LogUI("Frequence_coupure = %.2f\nGain_fc
283             =%.2f\nVout_fc=%.2f\n ",fc,gout_fc,vout_fc_rms);
284
285         if (count < 2)
286         {
287             free(freqs); free(gains);
288             MessagePopup("Erreur", "Pas assez de points re us.");
289             LogUI("Pas assez de points.");
290             return 0;
291         }
292
293         DeleteGraphPlot(panel, PANEL_GRAPH, -1,
294             VAL_IMMEDIATE_DRAW);
295         PlotXY(panel, PANEL_GRAPH,
296             freqs, gains, count,
297             VAL_DOUBLE, VAL_DOUBLE,
298             VAL_THIN_LINE, VAL_EMPTY_SQUARE,
299             VAL_SOLID, 1, VAL_RED);
300
301         double fc_marker[1];
302         double gain_marker[1];
303         fc_marker[0] = fc;
304         gain_marker[0] = gout_fc;
305
306         PlotXY(panel, PANEL_GRAPH,
```

```
305         fc_marker, gain_marker,
306         1,
307         VAL_DOUBLE, VAL_DOUBLE,
308         VAL_SCATTER, VAL_SOLID_CIRCLE,
309         VAL_SOLID, 10, VAL_GREEN);
310
311     }
312     LogUI("Trace OK (%d points).", count);
313     free(freqs);
314     free(gains);
315     return 0;
316 }
```